



Security Audit

Quant.Fun (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	12
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Quant.Fun team partnered with Hashlock to conduct a security audit of their HyperVaults smart contracts. Hashlock manually and proactively reviewed the code in order to ensure these contracts are secure for the project's team and clients.

Project Context

Quant.Fun (HyperVaults) is an onchain hedge fund platform on HyperLiquid that lets LPs deposit USDC into EIP-4626 vaults managed by authorized trading agents via HyperCore. The audited VaultImplv1 smart contract handles deposit/withdrawal flows, non-transferable share accounting and a management + performance fee system with a two-stage withdrawal queue. The primary use case is to serve institutional traders including hedge funds.

Project Name: Quant.Fun

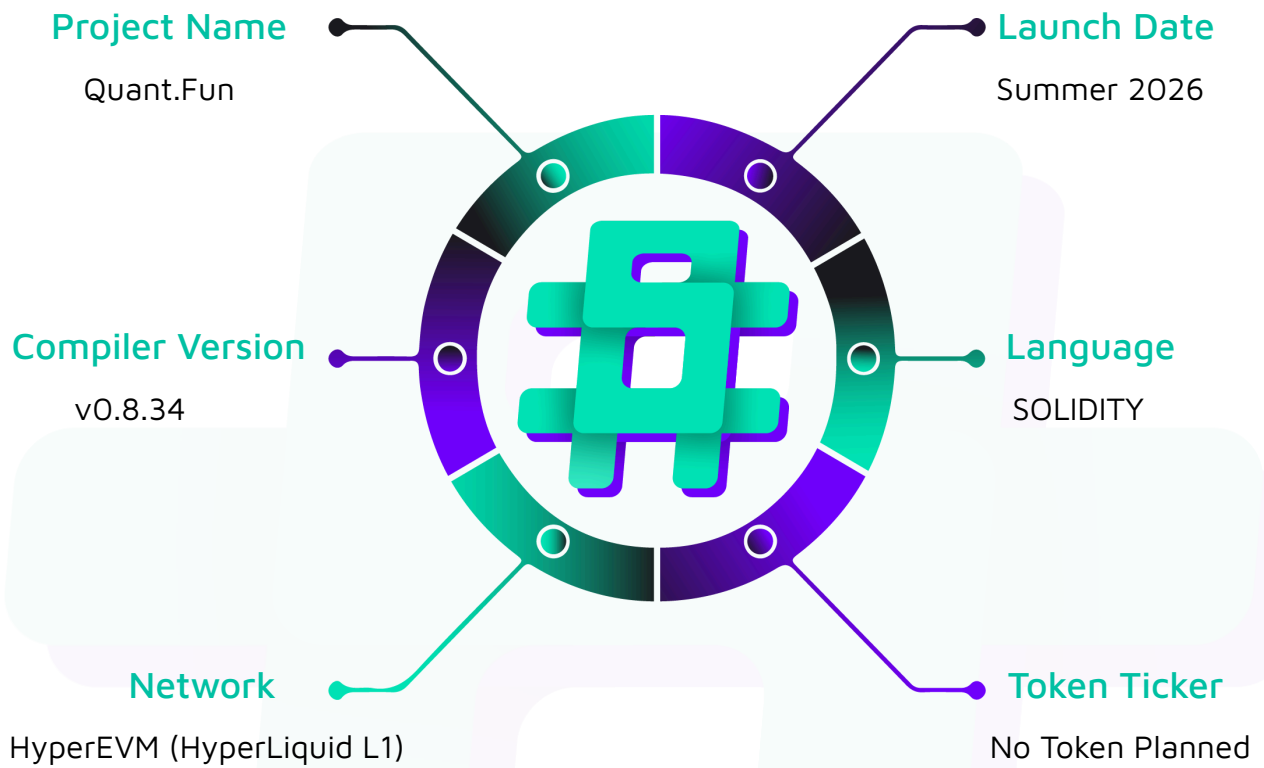
Project Type: DeFi

Compiler Version: 0.8.34

Website: <https://quant.fun/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the Quant.Fun project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Quant.Fun Smart Contracts
Network	HyperEVM (HyperLiquid L1)
Language	Solidity
Audit Date	May, 2026
Contract 1	VaultImplv1.sol
Audited GitHub Commit Hash	4fccb7f2a3f996dbf0bdf58a7ace29e1459053c4
Fix Review GitHub Commit Hash	dd6739cbc9316a6a3524fde78c791c091371659a

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Vulnerable"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin and Hyper EVM Lib contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium severity vulnerabilities

1 Low severity vulnerabilities

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
VaultImplv1.sol - Allows the admin (DEFAULT_ADMIN_ROLE) to: - Grant and revoke all roles - Allows the manager (MANAGER_ROLE) to: - Set fee recipient, management fee, and performance fee - Halt and clear deposits and withdrawals - Register and remove spot assets - Authorize API trading wallets - Set price deviation band - Allows the keeper (KEEPER_ROLE) to: - Execute pending withdrawals - Sweep USDC between spot and perp accounts - Bridge USDC from HyperCore to EVM - Run performance fee batch cranks - Allows the emergency role (EMERGENCY_ROLE) to: - Halt deposits and withdrawals for 48 hours - Allows LPs (users) to: - Deposit USDC and receive non-transferable shares - Request withdrawals (lock shares) - Cancel pending withdrawal requests	Contract achieves this functionality. However, the performance fee self-crank in <code>cancelWithdraw</code> uses the live account-wide HWM instead of the frozen snapshot, enabling a fee bypass when combined with a batch crank during a pending request.

Code Quality

This audit scope involves the smart contracts of the Quant.Fun (HyperVaults) project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Quant.Fun (HyperVaults) project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] VaultImplv1#cancelWithdraw - Performance fee bypass via Cancel-After-Crank HWM desync

Description

A depositor can avoid performance fees on the majority of their position by exploiting an inconsistency between how `applyProfitShareBatch` advances the account-wide `highWaterMark` and how `cancelWithdraw` reads it during the self-crank.

Vulnerability Details

When a depositor has a pending withdrawal, `applyProfitShareBatch` correctly charges fees only on the free shares (`balanceOf - lockedShares`). However, `_settlePerfFee` with `advanceHwm = true` pushes the entire account's `info.highWaterMark` to `currentPrice` - not just the portion that was actually fee'd.

Later, when the depositor calls `cancelWithdraw`, the self-crank in that function reads the live `info.highWaterMark` (already advanced to the peak by the batch crank). If the current price has retraced below that peak, `_settlePerfFee` hits the early return (`price <= hwm -> return 0`) and charges zero fees on the entire position - including the locked shares that never paid.

The `executeWithdraw` path is immune because it uses the frozen `req.hwmSnapshot` captured at request time. The cancel path does not.

The attacker's optimal strategy:

1. Lock ~99% of shares via `requestWithdraw`, leaving a tiny free portion.
2. Wait for the keeper to run `applyProfitShareBatch` - fees are charged on the ~1% free portion and HWM is advanced to the peak for the whole account.
3. Wait for any price retracement below the new HWM.

4. Call `cancelWithdraw` - self-crank sees `price ≤ hwm`, charges nothing. The ~99% locked portion's profit is never fee'd.

This is repeatable, requires no price manipulation and the attacker controls all parameters (lock fraction, timing of request and cancel).

```
function cancelWithdraw() external nonReentrant {

    VaultStorage storage $ = _getVaultStorage();

    address owner = msg.sender;

    WithdrawRequest memory req = $.withdrawRequests[owner];

    if (req.shares == 0) {
        revert NoWithdrawRequest();
    }

    _collectManagementFee($);

    if (!req.isFeeExempt) {
        _settlePerfFee(
            $,
            owner,
            balanceOf(owner),
            _pricePerShare(),
            $.depositors[owner].highWaterMark, // @audit uses live HWM, not
req.hwmSnapshot
            $.performanceFeeBps,                // @audit uses live fee rate, not
req.perfFeeBpsSnapshot
            true
        );
    }
}
```

```

    }

    // forge-lint: disable-next-line(unsafe-typecast)
    $.depositors[owner].lockedShares -= uint128(req.shares);

    delete $.withdrawRequests[owner];

    emit WithdrawCancelled(owner, req.shares);
}

```

Impact

Direct loss of performance fee revenue for the feeRecipient. The bypassed amount equals the performance fee that should have been charged on the locked shares gains from their original HWM to the current price. Scales linearly with position size, lock fraction and price appreciation. On a \$10M vault position with 50% performance fee and a 50% appreciation cycle, the bypassed fees can reach hundreds of thousands of dollars.

Recommendation

Split the cancel self-crank into two settlements and use the same snapshot semantics as executeWithdraw for the locked portion:

```

if (!req.isFeeExempt) {
    uint256 currentPrice = _pricePerShare();

    uint256 feePrice = req.snapshotPrice > currentPrice ? req.snapshotPrice :
currentPrice;

    DepositorInfo storage info = $.depositors[owner];

    uint256 freeShares = balanceOf(owner) - info.lockedShares;

    // Free shares: settle against live HWM (same as batch crank)
}

```

```
        _settlePerfFee($, owner, freeShares, currentPrice, info.highWaterMark,
$.performanceFeeBps, true);

        // Locked shares: settle against frozen HWM at max(snap, current) (same as
executeWithdraw)

        _settlePerfFee($, owner, req.shares, feePrice, req.hwmSnapshot,
req.perfFeeBpsSnapshot, false);

        // Advance HWM to highest settlement price - prevents both regression (partial lock)
// and double-charging on future cranks (full lock where batch never advanced HWM).

        if (feePrice > info.highWaterMark) {

            info.highWaterMark = uint128(feePrice);

        }

    }
}
```

Status

Resolved

Low

[L-01] VaultImplv1#addSpotAsset - No validation that `spotIndex` base token matches `tokenIndex`

Description

`addSpotAsset()` accepts `tokenIndex`, `spotIndex` and `perpIndex` as independent parameters but never verifies they refer to the same underlying asset.

Vulnerability Details

The `addSpotAsset()` function checks that the oracle is nonzero and the spot/oracle prices are within the deviation band at registration time. But it doesn't verify that `spotInfo(spotIndex).tokens[0] == tokenIndex`. A mismatched `spotIndex` would cause `_spotUsdValue6dec` to price one token's balance using another token's spot price, silently corrupting the AUM in `totalAssets()`. Two unrelated assets at similar prices would pass the band check at registration.

```
function addSpotAsset(uint64 tokenIndex, uint64 spotIndex, uint32 perpIndex) external
onlyRole(MANAGER_ROLE) {

    VaultStorage storage $ = _getVaultStorage();

    if ($.spotAssetPosition[tokenIndex] != 0) {
        revert SpotAssetAlreadyRegistered();
    }

    // weiDecimals >= szDecimals is required by _spotUsdValue6dec; an inverted token
would

    // cause a checked-arithmetic underflow in totalAssets(), DoSing the vault.

    PrecompileLib.TokenInfo memory info = PrecompileLib.tokenInfo(tokenIndex);
    require(info.weiDecimals >= info.szDecimals, "invalid token decimals");

    if (PrecompileLib.oraclePx(perpIndex) == 0) revert InvalidPerpIndex();

    SpotAsset memory sa = SpotAsset({tokenIndex: tokenIndex, spotIndex: spotIndex,
perpIndex: perpIndex});
```



```
// Reject any asset whose spot price is outside the configured band against perp
oracle.

if (!_assetWithinBand(sa, $.priceDeviationBandBps)) {

    revert PriceDeviationOutOfBand();

}

$.approvedSpotAssets.push(sa);

$.spotAssetPosition[tokenIndex] = $.approvedSpotAssets.length; // 1-indexed

emit SpotAssetAdded(tokenIndex, spotIndex, perpIndex);

}
```

Impact

A manager misconfiguration would produce incorrect AUM valuation, affecting share pricing for all deposits and withdrawals.

Recommendation

We recommend the adding of a cross-validation check.

Status

Resolved

QA

[Q-01] VaultImplv1#haltWithdrawals - Can be called repeatedly to extend halt beyond documented 7-day cap

Description

Short description with proper formatting. README 7.3 states: "The 7-day cap is hard - there is no in-contract path to extend a withdrawal halt beyond it." The code does not enforce this.

Vulnerability Details

haltWithdrawals computes `until = uint48(block.timestamp) + duration` and overwrites `withdrawalsHaltedUntil` on every call. Nothing prevents calling it before the current halt expires. A manager can call on day 1 (`until = day 8`), again on day 6 (`until = day 13`), again on day 12 (`until = day 19`), maintaining a continuous halt indefinitely. Each call is on-chain visible and ADMIN can revoke `MANAGER_ROLE`, but the documented guarantee is inaccurate.

```
/**
 * @notice Halt withdrawals for up to 7 days (MANAGER_ROLE only).
 * @dev EMERGENCY_ROLE is intentionally excluded from halting withdrawals.
 */
function haltWithdrawals(uint48 duration) external onlyRole(MANAGER_ROLE) {
    if (duration > MANAGER_WITHDRAW_HALT_MAX) revert HaltDurationTooLong();
    uint48 until = uint48(block.timestamp) + duration;
    _getVaultStorage().withdrawalsHaltedUntil = until;
    emit WithdrawalsHaltActivated(until);
}
```

Impact

LPs relying on the documentation's 7-day hard cap guarantee are misled. The actual enforcement is a per-call cap, not an absolute cap.

Recommendation

We recommend to either update the documentation to accurately describe the behavior or enforce an absolute cap by preventing re-calls before the current halt expires.

Status

Resolved

[Q-02] VaultImplv1#bridgeToEvm - NatSpec specifies 6-dec but mainnet requires 8-dec input

Description

The `bridgeToEvm` function's NatSpec says `@param amount 6-dec EVM units` but mainnet behavior requires 8-decimal input.

Vulnerability Details

README §14 documents this as a known mainnet finding: "vault.bridgeToEvm(amount) takes 8-decimal input on mainnet, not 6-decimal as the NatSpec suggests." The root cause is `HLConversions.evmToWei` performing a direct cast for USDC, passing the raw value to `sendAsset` which operates in HyperCore spot denomination (8-dec). The NatSpec was never updated to reflect this.

```
/**
 * @notice Bridge USDC from HyperCore spot → EVM (Action 13: sendAsset).
 *
 * @dev Second step of the withdrawal bridge-back. Requires vault to hold HYPE on
 * HyperCore for gas.
 *
 * After this call, USDC lands in balanceOf(address(this)) and executeWithdraw()
 * can proceed.
 *
 * @param amount 6-dec EVM units.
 */
function bridgeToEvm(uint256 amount) external onlyRole(KEEPER_ROLE) {
    CoreWriterLib.bridgeToEvm(HLConstants.usdc(), amount);
    emit BridgedToEvm(amount);
}
```

Impact

Any keeper implementation or future integration reading the contract's NatSpec would use 6-dec values, bridging 100× less USDC than intended and causing `executeWithdraw` to revert on insufficient balance.

Recommendation

We recommend updating the NatSpec to match the actual mainnet behavior:.

Status

Resolved



[Q-03] Missing `withdrawDelay` public view function**Description**

The `withdrawDelay` storage variable has no corresponding view function, unlike every other configuration parameter in the contract.

Vulnerability Details

All config params have public views: `feeRecipient()`, `managementFeeBps()`, `performanceFeeBps()`, `depositsHaltedUntil()`, `withdrawalsHaltedUntil()`, `priceDeviationBandBps()`. But `withdrawDelay`, which directly determines how long LPs must wait between `requestWithdraw` and `executeWithdraw` can only be read via raw storage slot probing.

Impact

LPs and integrators cannot verify the withdrawal delay through the contract interface. UI frontends and monitoring tools must reverse-engineer the ERC-7201 storage layout to display this value.

Recommendation

We recommend adding the missing view function for `withdrawDelay`.

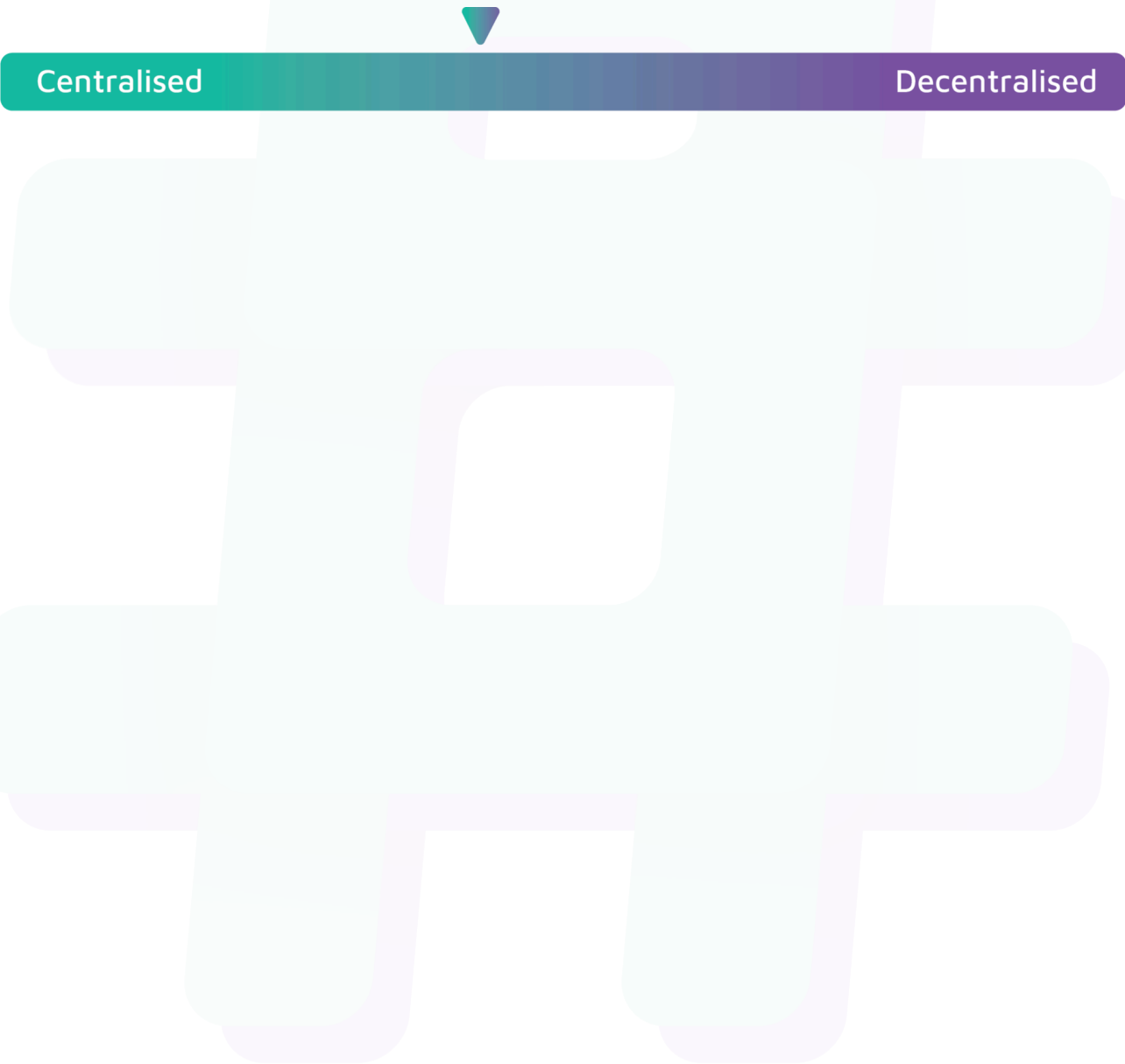
Status

Resolved

Centralisation

The Quant.Fun project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, Quant.Fun (HyperVaults) seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.